AWS Resource Tracker using Shell Scripting on EC2 (Live DevOps Project)

INTRODUCTION-This project focuses on automating the tracking of AWS resources using Shell Scripting and AWS CLI on a Linux EC2 instance. The objective of this project is to create a live DevOps automation script that can fetch and display AWS resource details such as EC2 instances, S3 buckets, IAM users, and Lambda functions.

The project demonstrates real-time cloud automation using AWS services, Linux commands, and Bash scripting, which are essential skills for DevOps and Cloud roles

PROJECT IMPLEMENTATION – STEP BY STEP PROCEDURE

Step 1: Login to AWS Management Console
First, open the AWS Management Console in the browser and log in using your AWS account credentials. After login, navigate to the AWS dashboard.

Step 2: Launch an EC2 Instance (Linux Server)

Go to the EC2 service from the AWS Console.

Click on Launch Instance

Provide the instance name as: linux project.

Select Amazon Linux 2023 as the operating system.

Choose the instance type .

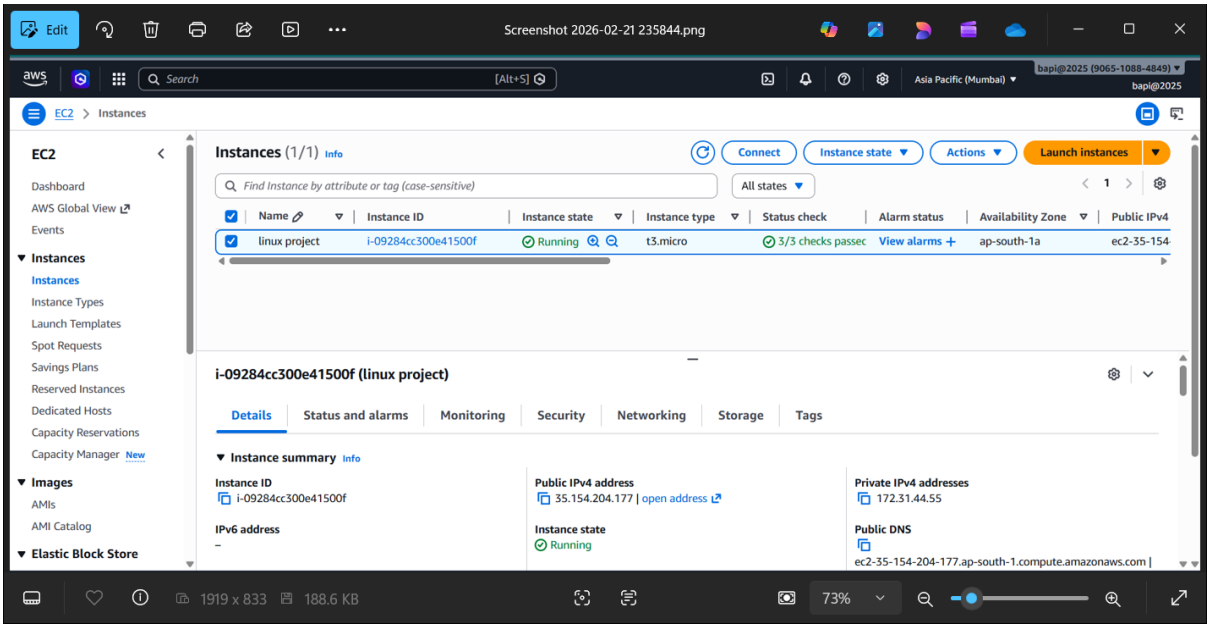
Create or select an existing key pair (linuxkey.pem).

Allow SSH (port 22) in the security group.

Click on Launch Instance.

Wait until the instance state shows “Running”.

Note the Public IPv4 address.



Step 3: Connect to EC2 Instance Using SSH (MobaXterm)

Open MobaXterm (SSH client).

Click on Session → SSH.

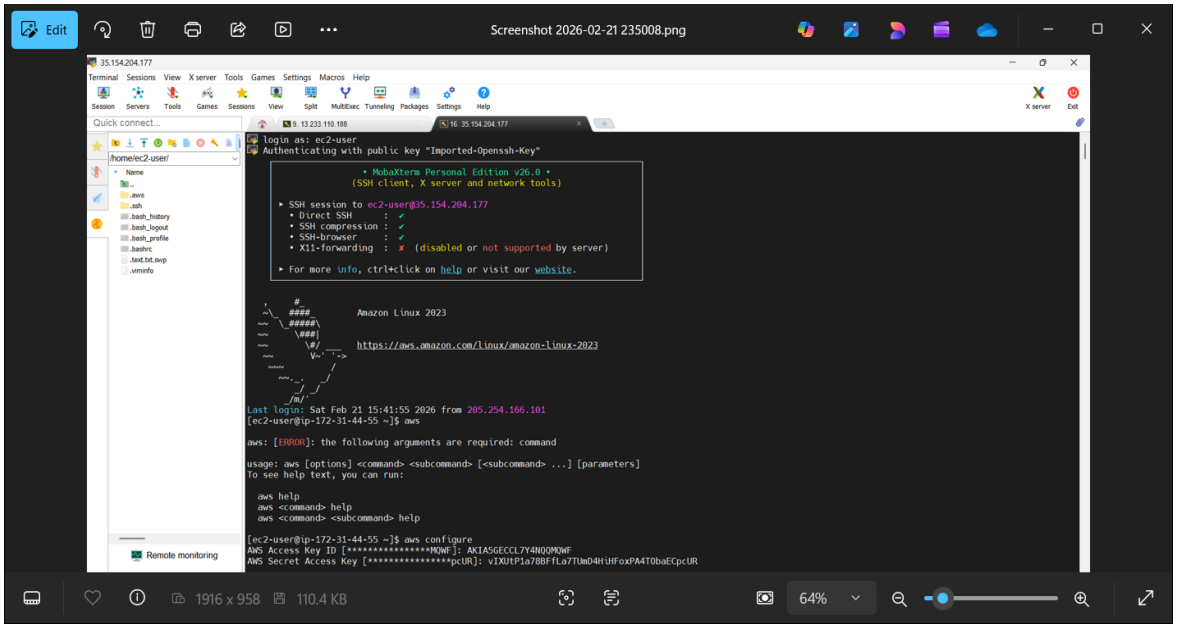
Enter the Public IP address of the EC2 instance.

Select the user as: ec2-user.

Browse and attach the private key file (linuxkey.pem).

Click OK to establish the SSH connection.

Successful login will display the Amazon Linux terminal.



Step 4: Verify AWS CLI is Installed on the Server
After logging into the EC2 instance, verify AWS CLI by typing:
aws --version

If AWS CLI is available, the version details will be displayed.
(This confirms the server can run AWS commands.)

Step 5: Create an IAM User for CLI Access

Go to IAM service in AWS Console.

Click on Users → Create User.

Enter username: cli-user.

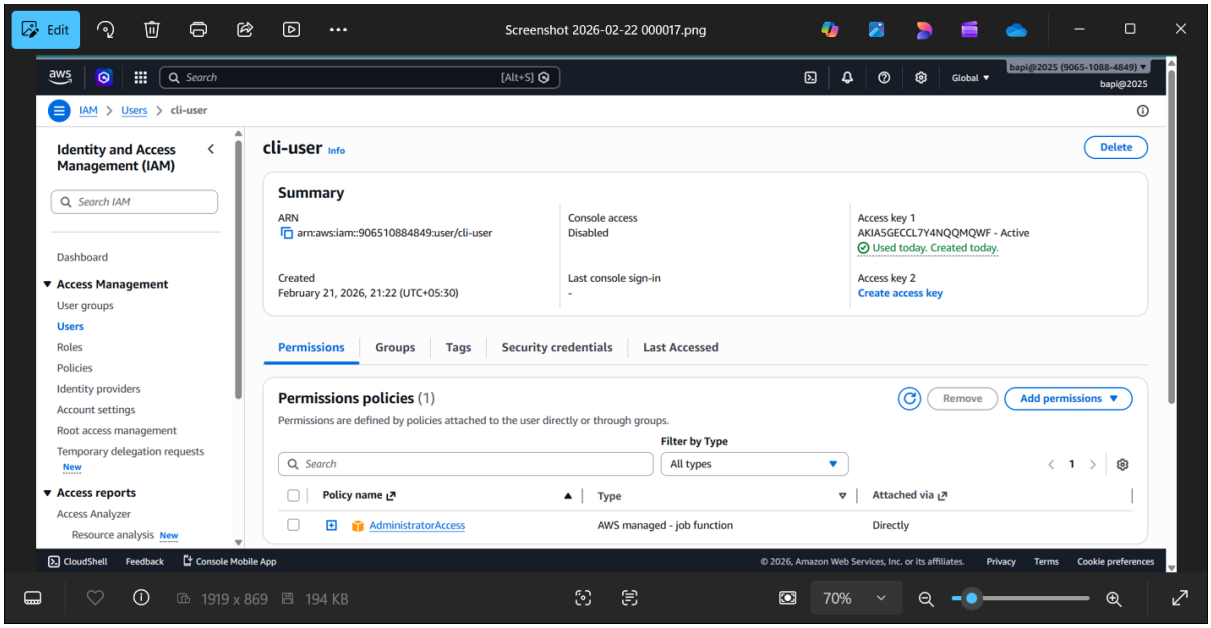
Enable Programmatic Access.

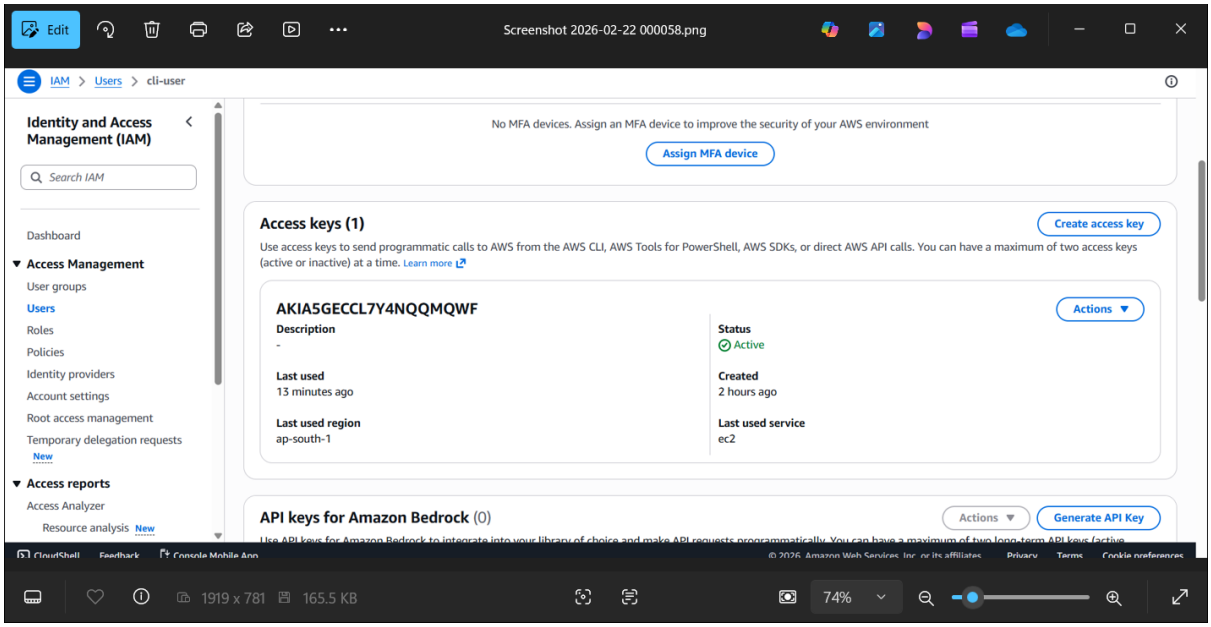
Attach permission policy: AdministratorAccess.

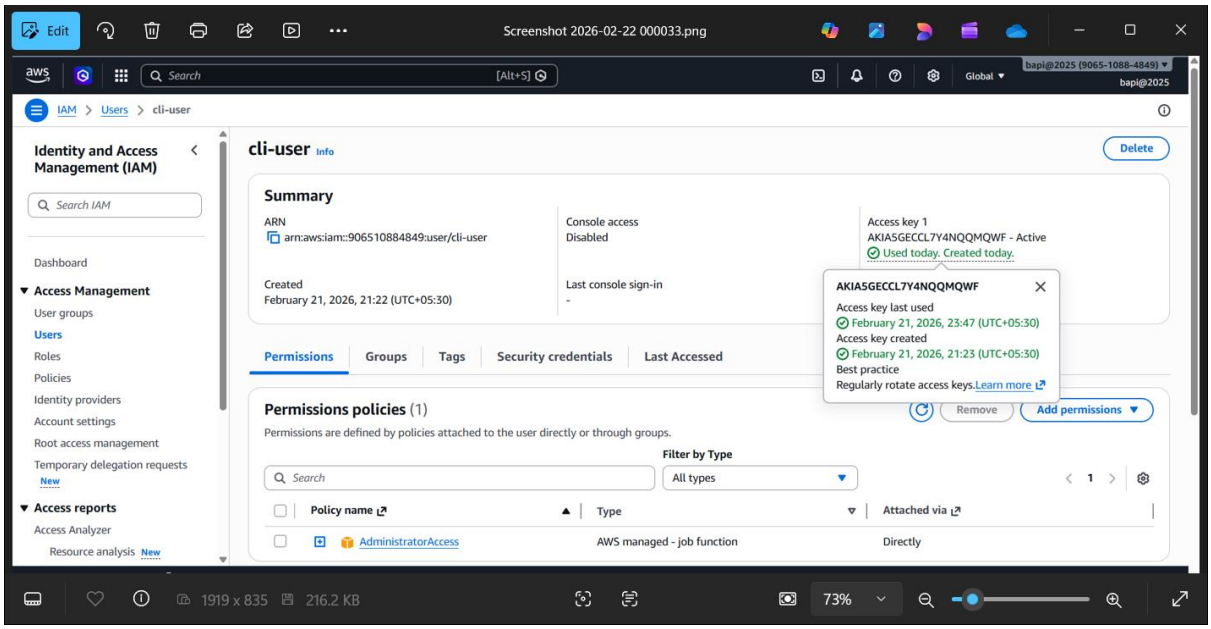
Complete the user creation process.

Generate Access Key ID and Secret Access Key.

Download or securely copy the credentials.




Screenshot 2026-02-22 000058.png
IAM > Users > cli-user
Identity and Access Management (IAM)
Search IAM
Dashboard
Access Management
User groups
Users
Roles
Policies
Identity providers
Account settings
Root access management
Temporary delegation requests
New
Access reports
Access Analyzer
Resource analysis New
No MFA devices. Assign an MFA device to improve the security of your AWS environment
Assign MFA device
Access keys (1)
Create access key
Use access keys to send programmatic calls to AWS from the AWS CLI, AWS Tools for PowerShell, AWS SDKs, or direct AWS API calls. You can have a maximum of two access keys (active or inactive) at a time. Learn more
AKIA5GECCL7Y4NQQMQWF
Actions
Description
-
Status
Active
Last used
13 minutes ago
Created
2 hours ago
Last used region
ap-south-1
Last used service
ec2
API keys for Amazon Bedrock (0)
Actions
Generate API Key
1919 x 781
165.5 KB
74%


Screenshot 2026-02-22 000033.png
Search
[Alt+S]
Global
bapi@2025 (9065-1088-4849)
bapi@2025
IAM > Users > cli-user
Identity and Access Management (IAM)
Search IAM
Dashboard
Access Management
User groups
Users
Roles
Policies
Identity providers
Account settings
Root access management
Temporary delegation requests
New
Access reports
Access Analyzer
Resource analysis New
cli-user Info
Delete
Summary
ARN
arn:aws:iam::906510884849:user/cli-user
Console access
Disabled
Access key 1
AKIA5GECCL7Y4NQQMQWF - Active
Used today. Created today.
Created
February 21, 2026, 21:22 (UTC+05:30)
Last console sign-in
-
AKIA5GECCL7Y4NQQMQWF
Access key last used
February 21, 2026, 23:47 (UTC+05:30)
Access key created
February 21, 2026, 21:23 (UTC+05:30)
Best practice
Regularly rotate access keys.Learn more
Permissions
Groups
Tags
Security credentials
Last Accessed
Permissions policies (1)
Remove
Add permissions
Permissions are defined by policies attached to the user directly or through groups.
Filter by Type
Search
All types
Policy name
Type
Attached via
AdministratorAccess
AWS managed - job function
Directly
1919 x 835
216.2 KB
73%

Step 6: Configure AWS CLI on EC2 Instance

Run the following command in the EC2 terminal:

aws configure

Then enter the required details:

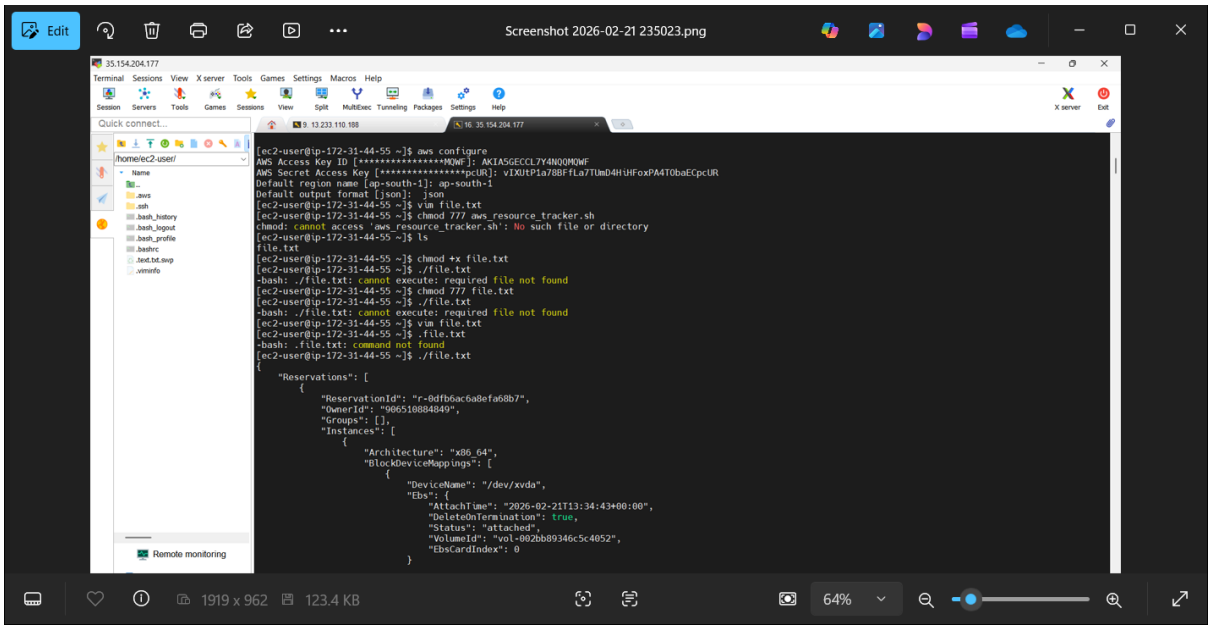
AWS Access Key ID: (Your IAM Access Key)

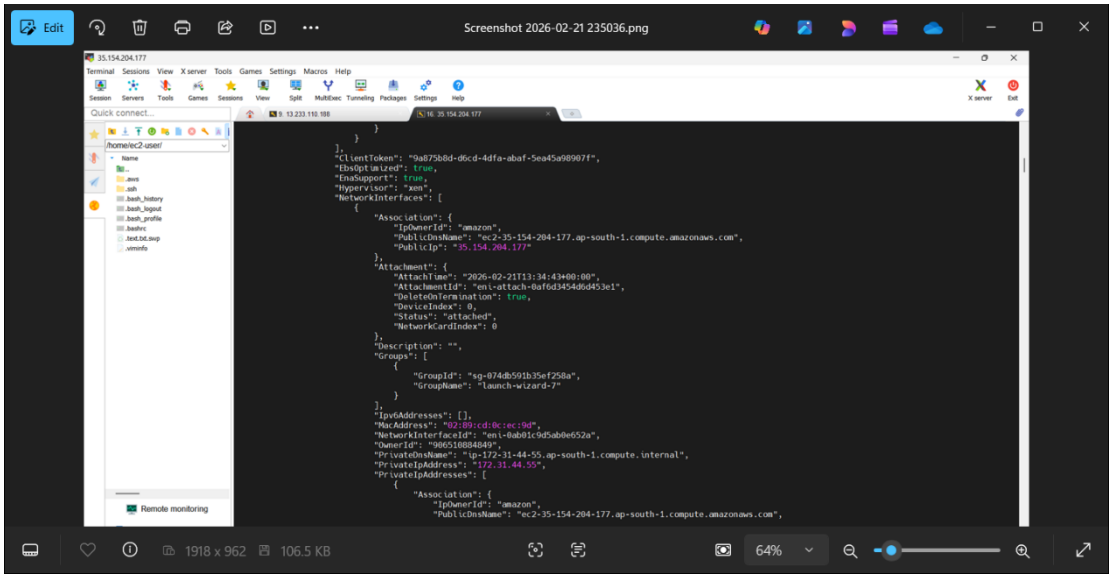
AWS Secret Access Key: (Your Secret Key)

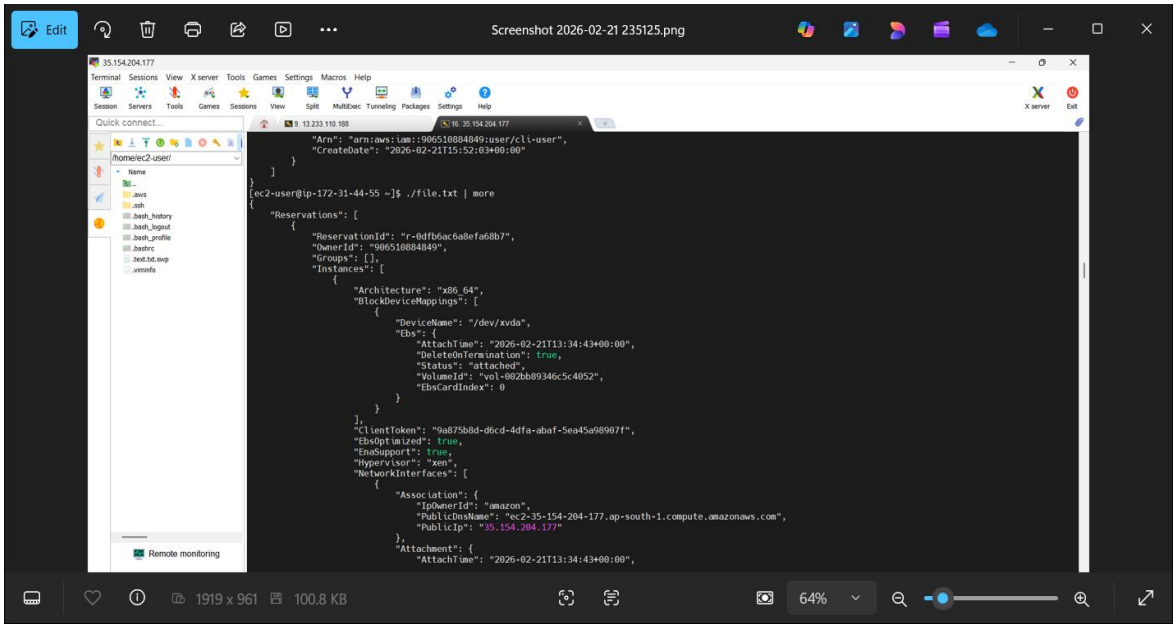
Default region name: ap-south-1

Default output format: json

After configuration, the EC2 instance will be authenticated to access AWS services using CLI.







Step 7: Create a Shell Script File

To create the script file, use the vim editor:

vim file.txt

Press “i” to enter insert mode and write the shell script that tracks AWS resources such as S3 buckets, EC2 instances, Lambda functions, and IAM users.

After writing the script:

Press ESC

Type :wq!

Press Enter to save and exit the file.

Step 8: Write the AWS Resource Tracker Shell Script
Inside the file, a Bash script was written with a shebang (#!/bin/bash) and AWS CLI commands to fetch resource information.

The script included:

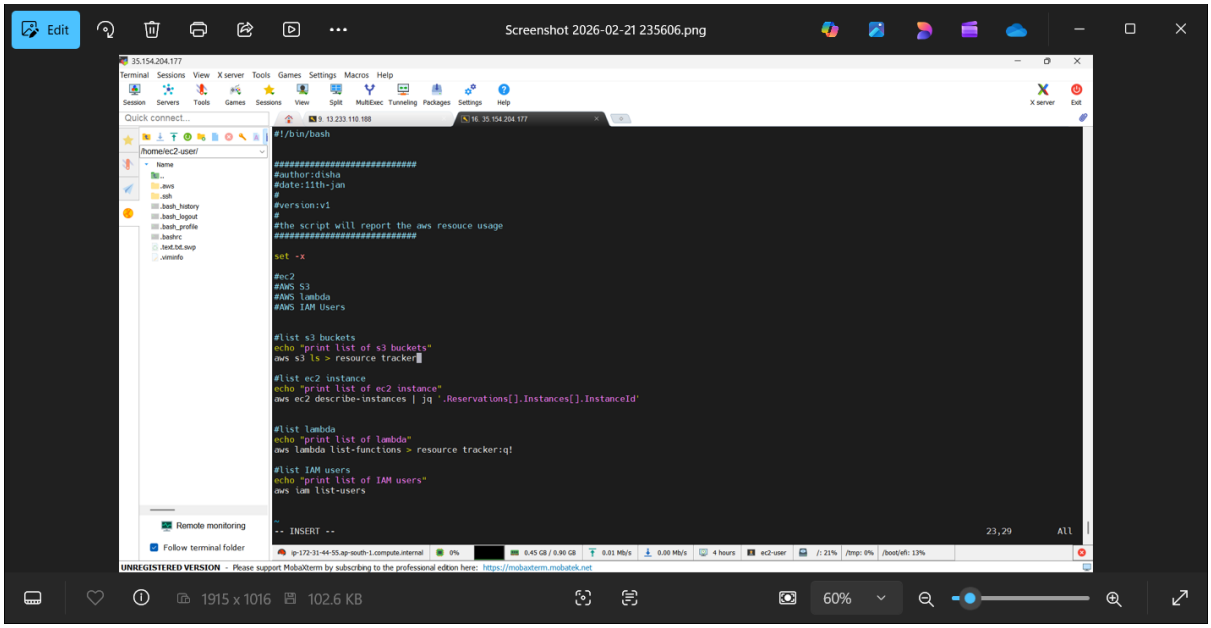
Listing S3 buckets

Describing EC2 instances

Listing Lambda functions

Listing IAM users

This script automates the process of monitoring AWS resources.



Step 9: Provide Execute Permission to the Script
By default, the file does not have execution permission.
So the following command was used:
chmod +x file.txt

This command makes the script executable in the Linux environment.

Step 10: Execute the Shell Script

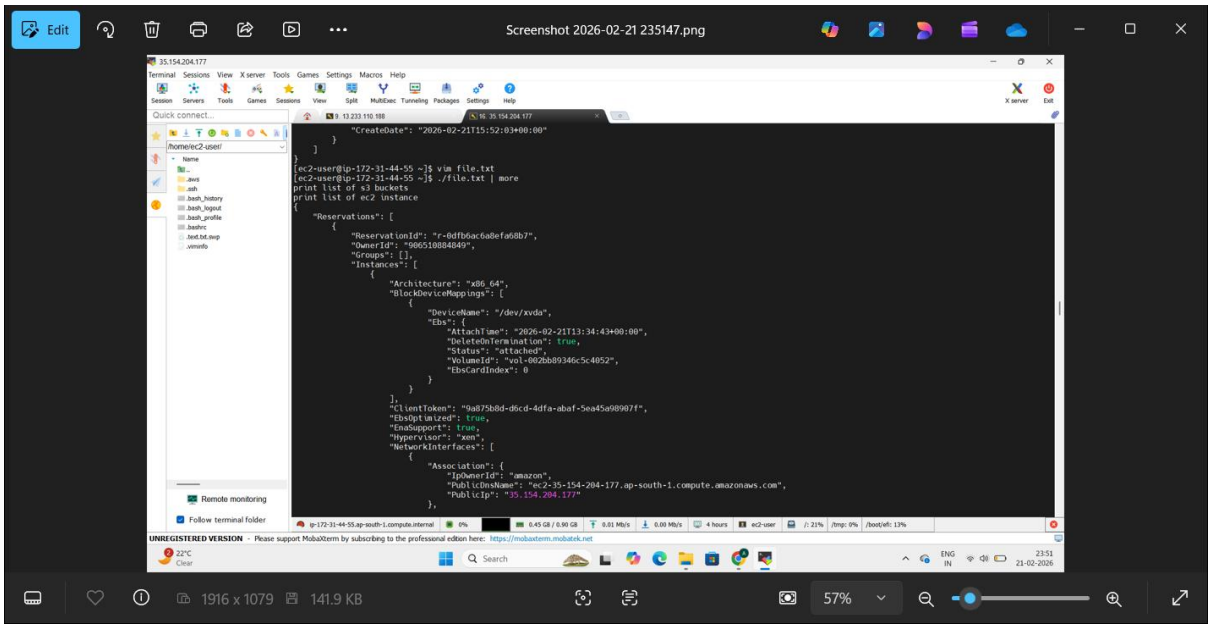
After giving execute permission, the script was executed using:

./file.txt

To view output page by page, the following command was also used:

./file.txt | more

This successfully ran the automation script on the EC2 server.



Verify Output and Resource Tracking

The script output confirmed:

Running EC2 instance details

Public and Private IP addresses

IAM user list (cli-user)

AWS resource usage information

Successful AWS CLI integration with the server

This verified that the automation script was working correctly.